



ADR-002 — The v1 store, and how the server orders songs

- **Status:** Accepted
- **Date:** 2026-09-05
- **Deciders:** Jay
- **Supersedes:** none. Extends ADR-001, which chose Go and the sync-first shape.

Context

Phase 2 needs somewhere to keep a scanned library and something to order it by. Two questions had to be settled before the HTTP API could be written, and both have answers that are easy to get wrong quietly.

1. **Where does the catalog live?** ADR-001 promised "no database server required for v1". That says what we will not do, not what we will.
2. **In what order does a browse request return songs?** A server that sorts differently from the client produces a list the player cannot recognise, and nothing about it looks broken — the songs are all there, in an order that is merely wrong.

Decision

1. The catalog is an in-memory index, rebuilt at start

`internal/library` walks the library once at startup, holds every entry in memory, and serves every request from that. There is no database, no file format, and nothing to migrate.

Why: the whole catalog for a large library is small. The 22-case corpus indexes in 15 ms; even a library of tens of thousands of songs is tens of megabytes of struct and a scan measured in seconds, on a Pi as much as on a workstation. Against that, a persisted catalog buys a faster start and costs a schema, a migration path, and a second source of truth that can disagree with the disk. Every one of those is a place for the server to be confidently wrong about what it holds.

What it costs, stated plainly: start-up time is proportional to library size, and a song added to the library is invisible until a rescan. Both are acceptable for v1 and both are fixable underneath this interface — `library.Store` already swaps a freshly built index in atomically, so a rescan never shows a half-built library to a request in flight.

Rejected: SQLite. The obvious choice, and the wrong one here for a specific reason: the cgo driver would break the cross-compilation that ADR-001 rests on — `linux/arm64` for the Pi from a Windows workstation is one `GOOS` / `GOARCH` away right now and would stop being so. The pure-Go driver avoids that but is a large dependency to carry for a query load of "filter a slice and sort it".

2. Packed archives are cached on disk, keyed by package hash

A song found as a loose folder has to be packed before it can be served as `.sng`. `internal/packcache` packs it once into `<data>/packs/<package_hash>.sng` and serves the file thereafter.

Why a file rather than streaming to the response: streaming means no `Content-Length`, no `Range`, and therefore no resume. A sync client that cannot resume a 40 MB song over a home wifi link is a sync client that starts again from zero, and Phase 2b is exactly that client.

The package hash is a sound key because it is a hash of the content: the same package produces the same archive, on any server, after any rescan. Archives are written to a temp file and renamed, so a crash or a full disk mid-pack cannot leave a truncated archive at the final name for every later request to serve as though it were whole.

3. Sorting reproduces the client's `SortString`, exactly

`internal/sortkey` is a faithful reimplementation of YARG.Core's `SortString` and the three transforms behind it, and `internal/library` orders by the twelve `SongAttribute` values using upstream's own comparer chains.

Why go to that trouble: the normalisation is not cosmetic. It drops a leading article, so "The Beatles" files under B. It folds diacritics, so "Björk" sorts beside "Blur" rather than after every ASCII name in the library. It strips Unity rich-text markup, so a chart titled `<color=#ff0000>Red</color>` does not file under `<`. A server that skipped any of this would return a list that is internally consistent and unlike anything the player sees in the game.

Three upstream behaviours are reproduced although each looks like a defect, because agreeing with the client is the point and "improving" it would mean disagreeing:

- Only uppercase `Æ` is expanded to `AE`; lowercase `æ` is untouched and sorts as non-ASCII.
- Comparison is by UTF-16 code unit, not code point, so an emoji sorts below `U+E000`.
- The article list is `the, el, la, le, les, los` — **not** `a` or `an`.

4. Two documented divergences from the client

Neither is a mistake and both are recorded here so they are not quietly "fixed" later.

The final tie-breaker is `package_hash`, not the entry's location. Upstream breaks a tie with `SortBasedLocation`, an absolute path on the player's own machine. That value would make the same library sort differently on two machines, and the server has no such path for a client in any case. `package_hash` is stable, content-derived and unique per package.

Genre, Subgenre, Source, Artist_Album and DateAdded have no comparer upstream — they are collected as grouping values rather than ordered. The chains used for those five are ours, and they are ours in the code comments too, rather than presented as parity.

5. Search matching is ours; only the folding is the client's

A query is folded through the same `SortString` normalisation, so a player who cannot type "Björk" still finds it. What happens next — a substring test across the eight text attributes — is this server's own. Upstream's own search logic has not been read, so no claim of parity is made for it, and the API documentation says so.

Consequences

Good

- No schema, no migration, no second source of truth about what the library contains.
- Cross-compilation to every promised platform is untouched; the only new dependency is `golang.org/x/text`, which is pure Go, for Unicode normalisation that the standard library does not provide.
- Browse order matches the game, which is the difference between a useful list and a correct-looking one.
- A packed archive is cached once and is byte-identical on every later request, which is what makes `ETag` and `Range` honest.

Bad / accepted

- Start-up cost grows with the library, and a new song needs a rescan to appear.
- Memory holds the whole catalog. Acceptable at v1 scale; the first library that makes it hurt is the signal to put something underneath `library.Store`, not to change the API.
- The pack cache grows without a bound or an eviction policy. It is content-keyed, so it is always safe to delete; a policy is a Phase 2 finishing task, not a design question.
- Reproducing `SortString` means it can drift from upstream, exactly as ADR-001 accepted for the format parsers. Mitigated the same way: by measuring against a real client.